

Quantum k nearest neighbors algorithm

Afrad Basheer^{1,2}, A. Afham^{1,3}, and Sandeep K. Goyal³

¹Centre for Quantum Software and Information, University of Technology, Sydney, Australia

²Chennai Mathematical Institute, H1 SIPCOT IT Park, Kelambakkam, Tamil Nadu 603103, India

³Department of Physical Sciences, Indian Institute of Science Education & Research (IISER) Mohali, Sector 81 SAS Nagar, Manauli PO 140306 Punjab India.

One of the simplest and most effective classical machine learning algorithms is the k nearest neighbors algorithm (k NN) which classifies an unknown test state by finding the k nearest neighbors from a set of M train states. Here we present a quantum analog of classical k NN – quantum k NN (Qk NN) – based on fidelity as the similarity measure. We show that the Qk NN algorithm can be reduced to an instance of the quantum k maxima algorithm; hence the query complexity of Qk NN is $O(\sqrt{kM})$. The non-trivial task in this reduction is to encode the fidelity information between the test state and all the train states as amplitudes of a quantum state. Converting this amplitude encoded information to a digital format enables us to compare them efficiently, thus completing the reduction. Unlike classical k NN and existing quantum k NN algorithms, the proposed algorithm can be directly used on quantum data, thereby bypassing expensive processes such as quantum state tomography. As an example, we show the applicability of this algorithm in entanglement classification and quantum state discrimination.

1 Introduction

Quantum machine learning (QML) [1, 2, 3, 4] is a recent offspring of quantum computing and machine learning (ML). One of the proposed applications of QML is using quantum computers to speed up ML tasks [5, 6]. In a converse manner, ML has proven itself adept at problems in

physics [7, 8, 9, 10]. We have also seen the realisation of quantum versions of several (classical) ML algorithms [11, 12, 13]. Along the same vein, we propose a quantum version of the k nearest neighbor algorithm [14] by reducing it to an instance of the quantum k maxima finding algorithm [15, 16] and providing an explicit construction of the oracle required. The algorithm is capable of classifying states without their explicit classical description. We then present two problems of interest as applications of our algorithm, namely entanglement classification and quantum state discrimination.

k NN is a simple supervised ML algorithm used extensively for pattern recognition and classification. This algorithm rests on the assumption that two points close to each other are more likely to be of the same type. The algorithm is provided with a set of train states (vectors) whose class labels are known. The test state (vector) with the unknown label is compared with the train states, and the k number of nearest neighbors of the train states are identified for the given test state. Finally, the label of the test state is determined upon majority voting.

The expensive step in the k NN algorithm is to determine the distance between the test state and all the train states. Each state (train or test) is represented by a vector of real (or complex) numbers. As the number of train states and the size of the state vectors increases, k NN becomes more expensive. To classify vectors of dimension N by comparing it to a set of train vectors of cardinality M , we need to carry out $O(MN)$ operations.

In this article, we propose a novel quantum k nearest neighbor (Qk NN) algorithm, a quantum analog of the classical k NN algorithm. Utilising existing algorithms such as the Swap test [17], k minima finding algorithm [15], quantum phase estimation, and the recent quantum analog-to-

Afrad Basheer: Afrad.M.Basheer@student.uts.edu.au

A. Afham: Afham@student.uts.edu.au

Sandeep K. Goyal: skgoyal@iisermohali.ac.in

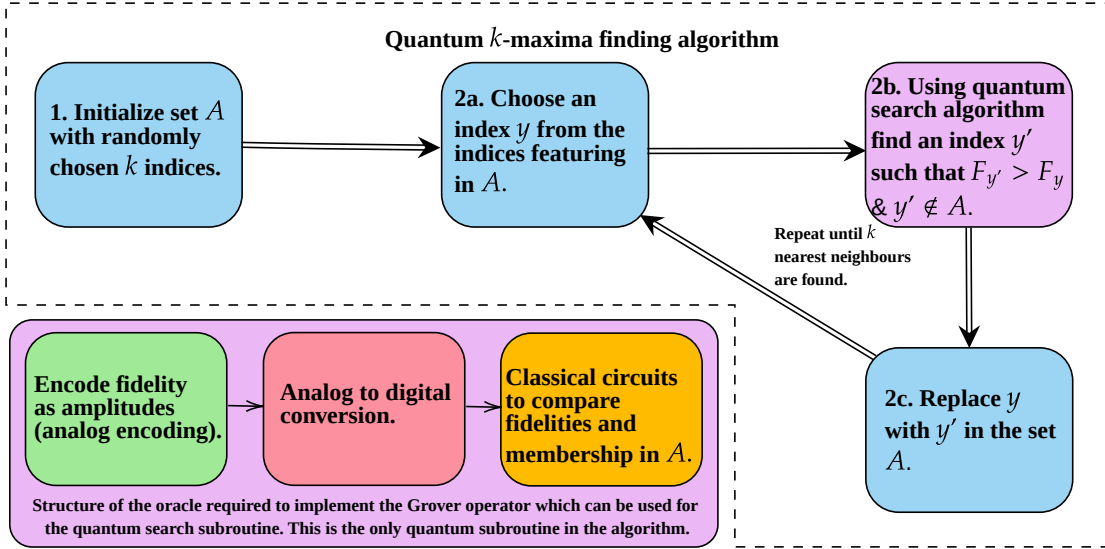


Figure 1: An overview of the QkNN algorithm. The idea is to use the quantum k maxima finding algorithm on a table $F = [F_0, \dots, F_{M-1}]$ to find the k nearest neighbors of the test state. Here $F_j = |\langle \psi | \phi_j \rangle|^2$ is the fidelity of the test state $|\psi\rangle$ with the j th train state $|\phi_j\rangle$. We start with a set A of randomly chosen k indices and then, using quantum search algorithm (Grover’s search when the number of targets are unknown), we replace indices in A with new indices (that are not in A) until the top k neighbors are found. The crucial step here is to prepare an oracle capable of performing the required Grover operator in the quantum search subroutine. The oracle should be capable of making comparisons of the form $F_{y'} > F_y$ and ensure that $y' \notin A$. Such an oracle is constructed by first extracting the fidelity using Swap test and encoding it as amplitudes of a quantum state. Then, an analog to digital conversion of the amplitudes is carried out which results in the fidelity being encoded as digital (bit-string) states. Once this is done, we perform a series of classical operations to compare the fidelities and check membership in A .

digital conversion algorithm [18], we construct an algorithm capable of classifying states without the requirement of having their classical description (writing out the amplitudes in some basis). This is particularly useful in cases where the data to be classified is inherently quantum, and thereby we can bypass expensive processes such as tomography [19] to learn the description of the state in question. We also provide two applications of our algorithm, namely entanglement classification of pure states and quantum state discrimination. Moreover, since we use quantum k maxima finding algorithm to find the k nearest neighbors, the query complexity of our algorithm is $O(\sqrt{kM})$.

Previous work in quantum versions of k NN include [20, 21, 22, 23, 24]. Ruan et al. [21] uses Hamming distance as the metric to estimate the k nearest neighbors. Dang et al. [23] uses quantum k nearest neighbors algorithm proposed in [22] for image classification. Of all the previous papers mentioned, the work by Chen et al. [22] seems to be the closest to ours with the same overall query complexity of $O(\sqrt{kM})$.

However, there are a few features that distinguish our work from the above-mentioned works. Firstly, we envision our algorithm to be used directly on quantum data (though it can also be used on ‘classical data’), which allows us to classify states without having their explicit classical description. Instead, we require the circuits capable of preparing the test state. Also, we use fidelity and dot product as a measure of similarity, and we demonstrate problems where fidelity can be used to carry out classification.

In Section 2, we present the necessary background, and in Section 3, we present our quantum k nearest neighbors algorithm, which uses fidelity as a measure of closeness. Then, in Section 4, we present a variation of the algorithm that uses dot product instead of fidelity for classification. Both of these algorithms build upon generalisations of algorithms from [18]. Section 5 comprises the query complexity of our algorithm which we show to be $O(\sqrt{kM})$. We then present in Section 6 two problems where our QkNN algorithm can be utilised - the problem of entanglement classification and a problem analogous to quantum state

discrimination.

Finally, we conclude in Section 7 while also discussing the kind of problems our algorithm would be adept at solving.

2 Background

In this section, we provide the relevant background for the Quantum k NN algorithm. We begin with the introduction of the classical k NN algorithm

2.1 Classical k NN algorithm

Let $\{u_n\}$ be a collection of vectors of unknown labels, which we call *test states*. The aim is to accurately assign these test states labels. For this, the k nearest neighbors (k NN) algorithm requires a collection of vectors $\{v_m\}$ of the same dimension whose labels are known to us. We shall call these states *train states*. k NN assigns labels to each test state by first computing the k nearest neighbors of the test state. Then a majority voting is carried out among these k nearest neighbors. Ties are resolved in different ways, such as assigning the label of the nearest training point or the label of a random training point among the k nearest neighbors. Successful applications of k NN include [25, 26]. Being a simple algorithm, k NN also allows us to reason about the structure of the data we are working with.

Let the test states and the train states are r -dimensional real or complex vectors. Any bona fide definition of a distance measure can be used for the purpose of k NN algorithm. Most common distance measures include Euclidean distance $d(\mathbf{u}, \mathbf{v}) = \sqrt{\sum_i^r |u_i - v_i|^2}$ and cosine similarity $c(\mathbf{u}, \mathbf{v}) = \frac{\langle \mathbf{u} | \mathbf{v} \rangle}{\|\mathbf{u}\| \cdot \|\mathbf{v}\|}$ (which reduces to inner product for normalised states), where $\mathbf{u}$ and $\mathbf{v}$ are r -dimensional complex vectors.

In quantum information, the fidelity function, though not a metric, $F(\cdot, \cdot)$ can be used to assign a notion of nearness between two quantum states belonging to the same space. For arbitrary quantum states ρ, σ belonging to the same space, the fidelity between them is $F(\rho, \sigma) = \left(\text{Tr} \left(\sqrt{\sqrt{\rho} \sigma \sqrt{\rho}} \right) \right)^2$. For pure quantum states $\rho = |u\rangle\langle u|, \sigma = |v\rangle\langle v|$, the fidelity function simplifies to $F(u, v) = |\langle u | v \rangle|^2$.

Though fidelity is not a metric, the metric Bures distance $B(u, v) = \sqrt{2 - 2\sqrt{F(u, v)}}$, defined

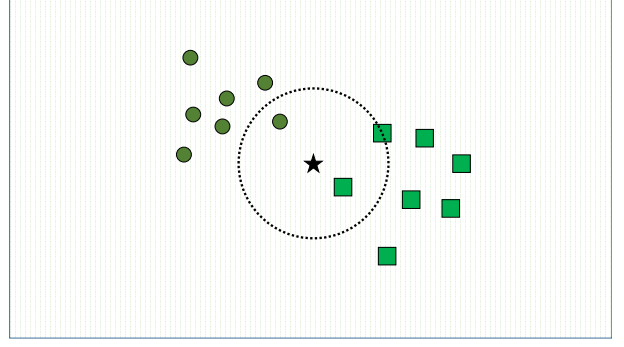


Figure 2: Choosing a $k = 3$ neighborhood. Here circle and square represents two different classes and star represents the unknown state whose label is to be determined. On choosing $k = 3$, we classify it as a ‘square’ point.

over quantum states, is a monotonous function of the fidelity. Therefore, finding the k -nearest neighbors of any quantum state with respect to the Bures distance is the same as finding the k states with the largest fidelity to the chosen quantum state.

The rationale behind k NN is that data points that are close together, with respect to some distance measure, must be similar. Formally, the k NN algorithm consists of the following steps (see Fig. 2):

1. For each test state (whose label is to be determined), compute its distance to the train states whose labels are known.
2. Choose the k number of neighbors which are nearest to the test state.
3. Assign the label using majority voting.

Although the k NN algorithm is simple to understand and easy to implement, the algorithm has its drawbacks. As the number of train data points and the dimension of the state vectors grows, k NN can quickly turn intractable. Classification of an N dimensional test state by comparing with M train states requires $O(MN)$ multiplication operations. Furthermore, there is no general way of choosing k , and usually, hyperparameter tuning is done to choose the best possible k [27].

2.2 Quantum k maxima finding algorithm

Durr and Hoyer describe, based on Grover search algorithm, an algorithm in [28] which can be used

to find the minimum of an unsorted list of size M with complexity $O(\sqrt{M})$. Through a simple modification, one can use the same algorithm to find the maximum instead of the minimum. A generalisation of the algorithm can be found in [15], which can be used to find the k smallest elements in a table $T = [T_0, \dots, T_{M-1}]$ of M elements in time $O(\sqrt{kM})$. A simple explanation of the algorithm can be found in [16].

The general idea of the algorithm is to start with randomly chosen k indices and use the quantum search algorithm [29] to find and replace the chosen indices with ones that have a higher table value. This process is repeated until we end up with the k highest values in the table. The algorithm is as follows:

1. Initialise a set $A = \{i_1, \dots, i_k\}$ with randomly chosen k indices from the list of M indices.
2. Repeat the following forever:
 - (a) Select threshold index y from A randomly.
 - (b) Using quantum search algorithm proposed in [29], find index y' which is not present in A , such that $T_{y'} > T_y$. This can be seen as using quantum search on the Boolean function

$$f_{y,A}(j) = \begin{cases} 1 & : T_j > T_y \text{ and } j \notin A, \\ 0 & : \text{otherwise.} \end{cases} \quad (1)$$

- (c) Replace y with y' in the set A .

The step 2b is the only quantum subroutine of the algorithm. Essentially, we use quantum search to find an index not present in A and has its table value greater than the table value of the threshold index. The quantum search algorithm is a slightly modified Grover's search algorithm, which is used in cases where the number of solutions to the boolean function is unknown. Note that this algorithm also uses the same Grover operator as in normal Grover's search. Also, instead of randomly sampling an index from A in step 2a, one can choose the index

$$y = \underset{i \in A}{\operatorname{argmin}} T_i.$$

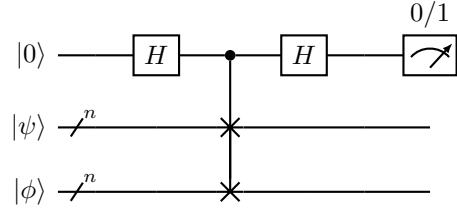


Figure 3: Circuit for Swap test.

This method would ensure a stopping criterion for the algorithm, that is, repeat step 2 until step 2b cannot be carried out. This is because if we cannot find an index, which is not in A and has a higher table value than the index with the minimum table value in A , then we should have all the k indices with the largest table values already present in A .

2.3 Swap Test

The swap test [17] is a quantum algorithm that can be used to statistically estimate the fidelity $F(\psi, \phi) = |\langle \psi | \phi \rangle|^2$ between two arbitrary n qubit pure states $|\psi\rangle$ and $|\phi\rangle$. The three register gate in Fig. 3 is the *controlled swap* (CSWAP) gate whose action is defined by

$$\begin{aligned} \text{CSWAP}|0\rangle|\psi\rangle|\phi\rangle &= |0\rangle|\psi\rangle|\phi\rangle, \\ \text{CSWAP}|1\rangle|\psi\rangle|\phi\rangle &= |1\rangle|\phi\rangle|\psi\rangle. \end{aligned} \quad (2)$$

To implement the swap test between states $|\psi\rangle$ and $|\phi\rangle$, we need three registers prepared in states $|0\rangle$, $|\psi\rangle$, and $|\phi\rangle$, respectively. The initial combined state of the three registers is $|0\rangle|\psi\rangle|\phi\rangle$. We then implement circuit in Fig. 3.

At the end of the circuit, the measurement probabilities of the first register are

$$\Pr(0) = \frac{1}{2} + \frac{1}{2}|\langle \psi | \phi \rangle|^2, \quad (3)$$

$$\Pr(1) = \frac{1}{2} - \frac{1}{2}|\langle \psi | \phi \rangle|^2. \quad (4)$$

The quantity $\Pr(0) - \Pr(1)$ gives us the desired fidelity.

2.4 Quantum Analog to Digital Conversion (QADC) algorithm

Mitarai et al. describe a set of algorithms in [18] to carry out analog to digital conversions within a quantum circuit. We provide a description of

what the algorithm does here and refer to Appendix (A) for details for the sake of brevity.

Let $\sum_{i=0}^{d-1} c_i |i\rangle$ be an arbitrary quantum state. Let $\{r_0, \dots, r_{d-1}\}$ be bitstrings that denote the best b -bit approximation of $\{|c_0|, \dots, |c_{d-1}|\}$ respectively. An m -bit abs-QADC algorithm can transform the *analog encoded* state $\sum_{i=0}^{d-1} c_i |i\rangle$ to the *digital encoded* state $\frac{1}{\sqrt{M}} \sum_{i=0}^{d-1} |i\rangle |r_i\rangle$.

Let $\{r_0, \dots, r_{d-1}\}$ be bitstrings that denote the best b -bit approximation of $\{\text{Re}(c_i), \dots, \text{Re}(c_{d-1})\}$ respectively. An m -bit real-QADC algorithm transforms the *analog encoded* state $\sum_{i=0}^{d-1} c_i |i\rangle$ and to the *digital encoded* state $\frac{1}{\sqrt{M}} \sum_{i=0}^{d-1} |i\rangle |r_i\rangle$.

In the coming sections, we show that variations to the circuits of these algorithms give rise to circuits capable of carrying out a quantum k NN algorithm.

3 Quantum k Nearest neighbors algorithm using fidelity

We now present the Quantum k Nearest neighbors algorithm. Refer Fig. 1 for an overview of the algorithm. Let $\mathcal{H}$ be the n -qubit Hilbert space of dimension $N = 2^n$ and let $|\psi\rangle \in \mathcal{H}$ be the unknown *test state* whose label is to be determined. Let

$$\{|\phi_j\rangle : j \in \{0, \dots, M-1\}\} \subset \mathcal{H} \quad (5)$$

be a collection of M *train states* whose labels are known to us. For the sake of convenience, we assume $M = 2^m$ for some positive integer m . The idea is to find the k nearest neighbors of $|\psi\rangle$ from the train states and then through majority voting, assign $|\psi\rangle$ a label. Let $F_j \equiv F(\psi, \phi_j) = |\langle \psi | \phi_j \rangle|^2$ be the fidelity between the test state $|\psi\rangle$ and the j^{th} train state $|\phi_j\rangle$ and define

$$F = [F_0, \dots, F_{M-1}] \quad (6)$$

to be a table of length M containing the fidelities with the test state $|\psi\rangle$ and all the train states $\{|\phi_j\rangle\}$.

3.1 A summary of the algorithm

Note that the problem of finding the k nearest neighbors of a test state $|\psi\rangle$ can be reduced to an instance of k maximum finding algorithm carried out on the table F given in Eq. (6). The only step that requires a quantum circuit in the quantum

k maximum finding algorithm is the quantum search subroutine given in step 2b. To achieve this, we should be able to prepare a circuit that carries out the oracle transformation

$$\mathcal{O}_{y,A} |j\rangle |0\rangle = |j\rangle |f_{y,A}(j)\rangle. \quad (7)$$

where $f_{y,A}$ is the Boolean function defined as

$$f_{y,A}(j) = \begin{cases} 1 & : F_j > F_y \text{ and } j \notin A, \\ 0 & : \text{otherwise.} \end{cases} \quad (8)$$

That is, the fidelity F_j must be greater than F_y and j should not feature in the threshold index set A . QkNN algorithm using fidelity as the similarity measure is then:

1. Using $\mathcal{O}_{y,A}$ as the required oracle for the Grover operator in quantum search in step 2b, use k maxima finding algorithm to find the k indices $\{j_1, \dots, j_k\}$ whose states $\{|\phi_{j_1}\rangle, \dots, |\phi_{j_k}\rangle\}$ have the maximum fidelity with the test state.
2. Conduct a majority voting among the k states and assign $|\psi\rangle$ the label of the majority.

The non-trivial part in the above steps is the realisation of the oracle $\mathcal{O}_{y,A}$. We briefly discuss this oracle in the next subsection and provide an explicit construction of it after that.

3.2 On the oracle $\mathcal{O}_{y,A}$

The construction of this oracle is based on the abs-QADC circuit from [18]. In that work, to compute the absolute values of the amplitudes of the state, we apply Swap test in superposition with the state and computational basis vectors. In the quantum k NN setting, we apply Swap test in superposition with the test state and the train states. Roughly speaking, the way to construct the oracle $\mathcal{O}_{y,A}$ is as follows:

1. Construct an operator $\mathcal{F}$ capable of the transformation

$$\mathcal{F} |j\rangle |0\rangle = |j\rangle |F_j\rangle \quad (9)$$

for arbitrary $j \in \{0, \dots, M-1\}$. Here $|F_j\rangle$ is the computational basis state which is the binary representation of F_j . This step can be broken down into two.

(a) First, we perform the transformation

$$\mathcal{E}^{\text{amp}}|j\rangle|0\rangle = |j\rangle|\Psi_j\rangle, \quad (10)$$

where $|\Psi_j\rangle$ is a state with information regarding F_j encoded in its amplitudes. We achieve this using the Swap test algorithm.

(b) We now perform the transformation

$$\mathcal{E}^{\text{dig}}|j\rangle|\Psi_j\rangle = |j\rangle|F_j\rangle. \quad (11)$$

This can be thought of as an analog to digital conversion as we are converting the fidelity information from amplitudes of $|\Psi_j\rangle$ to a digital format $|F_j\rangle$. We use a slightly modified version of the abs-QADC algorithm to achieve this.

Thus, $\mathcal{F} = \mathcal{E}^{\text{dig}}\mathcal{E}^{\text{amp}}$.

2. Consider two pairs of registers, *index*, *fidelity*; *index'*, *fidelity'* initialised as $|j\rangle_{\text{in}}|0\rangle_{\text{fid}}|y\rangle_{\text{in}'}|0\rangle_{\text{fid}'}$.

Apply $\mathcal{F}$ (step 1) on each of the two pairs of registers.

$$\xrightarrow{\mathcal{F}_{\text{in},\text{fid}}\mathcal{F}_{\text{in}',\text{fid}'}} |j\rangle_{\text{in}}|F_j\rangle_{\text{fid}}|y\rangle_{\text{in}'}|F_y\rangle_{\text{fid}'}. \quad (12)$$

where $\mathcal{F}_{\text{in},\text{fid}}$ denotes the gate $\mathcal{F}$ applied on the index and fidelity registers.

3. Now that we have our information in digital format, we may use a series of classical gates to realise the function $f_{y,A}$ (8). Note that any classical operation can be simulated in a quantum setting using Toffoli gates (refer page 29 of [30]). Let $\mathcal{C}$ denote the operator achieving (8). Uncomputing the irrelevant registers, we obtain

$$\xrightarrow{\mathcal{C},\text{uncompute}} |j\rangle|f_{y,A}(j)\rangle. \quad (13)$$

For the sake of brevity, we do not further expand upon the form of $\mathcal{C}$, just like with other gates, here.

We now provide an explicit construction of the oracle $\mathcal{O}_{y,A}$.

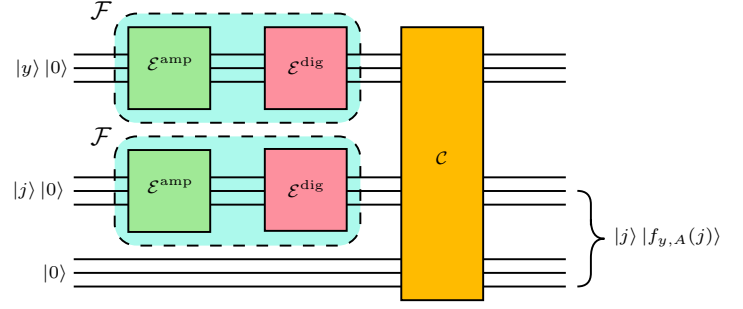


Figure 4: A breakdown of the oracle $\mathcal{O}_{y,A}$. In two pairs of registers corresponding to index j and y , apply $\mathcal{F}$ separately to obtain digital encoding of F_j and F_y . Using classical circuits (through Toffoli gates) compare F_j and F_y , and flip an ancilla qubit from $|0\rangle$ to $|1\rangle$ if $F_j > F_y$. Uncomputing irrelevant registers, we obtain $|j\rangle|f_{y,A}(j)\rangle$. Note that in the explicit construction of the oracle, there is an uncomputation procedure of one $\mathcal{F}$ coming in between the required classical operations. But for simplicity of explaining the circuit, we have avoided it in this diagram.

3.3 Constructing the oracle $\mathcal{O}_{y,A}$

We begin with the assumption that we are provided with efficient circuits of *state preparation* oracles $\mathcal{V}, \mathcal{W}$ of the form

$$\mathcal{V}|0^n\rangle = |\psi\rangle, \quad (14)$$

$$\mathcal{W}|j\rangle|0^n\rangle = |j\rangle|\phi_j\rangle, \quad (15)$$

for $j \in \{0, \dots, M-1\}$. That is, we do not require the classical description of the test state. Instead we require the circuits. Efficient implementation of oracles like $\mathcal{W}$ is discussed in [31]. Assuming such oracles are provided, we now describe the construction of the oracle $\mathcal{O}_{y,A}$.

1. Initialise four registers named *index*, *train*, *test*, B of sizes $m, n, n, 1$ respectively, where $n = \log N$ and $m = \log M$.

$$|j\rangle_{\text{in}}|0^{\otimes n}\rangle_{\text{tr}}|0^{\otimes n}\rangle_{\text{tst}}|0\rangle_B. \quad (16)$$

2. Apply $\mathcal{W}$ on train register

$$\xrightarrow{\mathcal{W}_{\text{in},\text{tr}}} |j\rangle_{\text{in}}|\phi_j\rangle_{\text{tr}}|0\rangle_{\text{tst}}|0\rangle_B. \quad (17)$$

3. Now apply $\mathcal{V}$ on test register to obtain

$$\xrightarrow{\mathcal{V}_{\text{tst}}} |j\rangle_{\text{in}}|\phi_j\rangle_{\text{tr}}|\psi\rangle_{\text{tst}}|0\rangle_B. \quad (18)$$

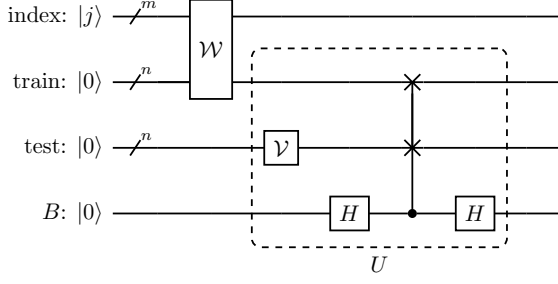


Figure 5: Circuit for steps 1-4.

4. Apply the swap test circuit (sans measurement) between train register and test register with B as the control qubit. The state is then

$$\begin{aligned}
 &\xrightarrow{\text{SwapTest}} \\
 &\frac{1}{2}|j\rangle_{\text{in}} \left[\left(|\phi_j\rangle_{\text{tr}} |\psi\rangle_{\text{tst}} + |\psi\rangle_{\text{tr}} |\phi_j\rangle_{\text{tst}} \right) |0\rangle_B \right. \\
 &\quad \left. + \left(|\phi_j\rangle_{\text{tr}} |\psi\rangle_{\text{tst}} - |\psi\rangle_{\text{tr}} |\phi_j\rangle_{\text{tst}} \right) |1\rangle_B \right] \\
 &\equiv |j\rangle_{\text{in}} |\Psi_j\rangle_{\text{tr,tst},B},
 \end{aligned} \tag{19}$$

where we have defined

$$\begin{aligned}
 |\Psi_j\rangle &= \frac{1}{2} \left[\left(|\phi_j\rangle_{\text{tr}} |\psi\rangle_{\text{tst}} + |\psi\rangle_{\text{tr}} |\phi_j\rangle_{\text{tst}} \right) |0\rangle_B \right. \\
 &\quad \left. + \left(|\phi_j\rangle_{\text{tr}} |\psi\rangle_{\text{tst}} - |\psi\rangle_{\text{tr}} |\phi_j\rangle_{\text{tst}} \right) |1\rangle_B \right].
 \end{aligned} \tag{20}$$

Define U to be the combined unitary transformations of steps 3 and 4 (refer Fig. 5). If one now measures the register B , one would see the probabilities as

$$\Pr(B = 0) = \frac{1 + F_j}{2}, \tag{21}$$

$$\Pr(B = 1) = \frac{1 - F_j}{2}. \tag{22}$$

The information regarding fidelity is now encoded in the amplitudes. Therefore gates from steps 2-4 makes up the $\mathcal{E}^{\text{amp}}$ operator given in Eq. (10). We must now convert it into a ‘digital’ format which can be further utilised.

5. To this end, construct a gate

$$G = U_{\text{tr,tst},B} \mathcal{W}_{\text{in,tr}} S_{0,\text{tr,tst},B} \mathcal{W}_{\text{in,tr}}^\dagger U_{\text{tr,tst},B}^\dagger Z_B, \tag{23}$$

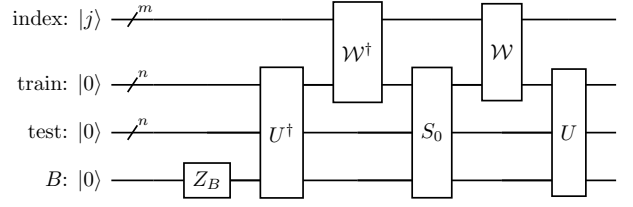


Figure 6: Constructing the operator G as defined in Eq. (23).

where Z_B denotes the application of the Z gate on register B and $S_0 = \mathbb{1} - 2|0\rangle\langle 0|$ (refer Fig. 6). This operator can be seen as the operator G used in the abs-QADC algorithm [18], with the CNOT gates replaced by the train data preparation oracle $\mathcal{W}$. The action of G on the current state can be written as controlled action of operators G_j (refer appendix F):

$$G|j\rangle_{\text{in}} |\Psi_j\rangle_{\text{tr,tst},B} = |j\rangle_{\text{in}} (G_j |\Psi_j\rangle_{\text{tr,tst},B}), \tag{24}$$

where

$$G_j = U_{\text{tr,tst},B} S_j U_{\text{tr,tst},B}^\dagger Z_B, \tag{25}$$

$$S_j = \mathbb{1} - 2(|\phi_j\rangle\langle\phi_j|_{\text{tr}} \otimes |0\rangle\langle 0|_{\text{tst},B}). \tag{26}$$

6. $|\Psi_j\rangle_{\text{tr,tst},B}$ can be decomposed into two eigenstates of G_j , namely $|\Psi_{j+}\rangle$ and $|\Psi_{j-}\rangle$, corresponding to the eigenvalues $e^{\pm i2\pi\theta_j}$, respectively. Here, $\sin(\pi\theta_j) = \sqrt{\frac{1}{2}(1 + F_j)}$ and $\theta \in [1/4, 1/2)$ (refer Appendix G). The decomposition is given as

$$|\Psi_j\rangle = \frac{-i}{\sqrt{2}} (e^{i\pi\theta_j} |\Psi_{j+}\rangle - e^{-i\pi\theta_j} |\Psi_{j-}\rangle), \tag{27}$$

7. We now have an operator G which has the fidelity value F_j stored in its eigenvalues. To get a b -bit binary representation of θ_j , we now run the quantum phase estimation algorithm (refer Appendix B for more details on quantum phase estimation) on G . To this end, we bring the *phase register* containing b qubits and run the phase estimation algorithm:

$$\begin{aligned}
 &\xrightarrow{\text{PhaseEst.}} \frac{-i}{\sqrt{2}} |j\rangle_{\text{in}} \left[e^{i\pi\theta_j} |\theta_j\rangle_{\text{ph}} |\Psi_{j+}\rangle_{\text{tr,tst},B} \right. \\
 &\quad \left. - e^{-i\pi\theta_j} |1 - \theta_j\rangle_{\text{ph}} |\Psi_{j-}\rangle_{\text{tr,tst},B} \right] \\
 &\equiv |j\rangle_{\text{in}} |\Psi_{j,\text{AE}}\rangle_{\text{ph,tr,tst},B}.
 \end{aligned} \tag{28}$$

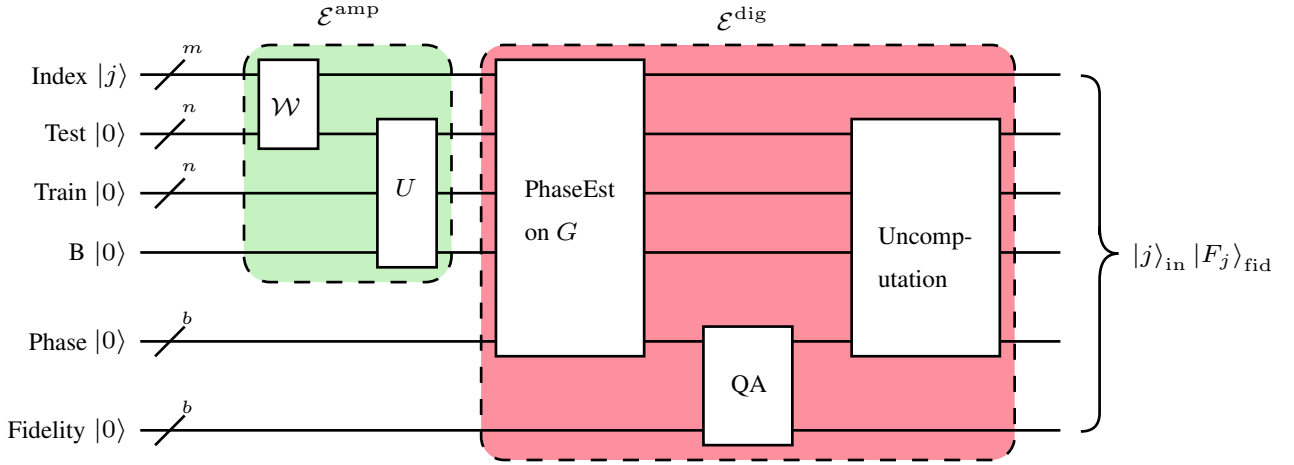


Figure 7: Detailed construction of the operator $\mathcal{F}$ defined in Eq. (9). We first use $\mathcal{E}^{\text{amp}}$ to encode the fidelity in the amplitudes. See Fig. (5) for details on U . Having the fidelity F_j encoded as amplitudes, we want to convert them into a digital format. To this end we use $\mathcal{E}^{\text{dig}}$, which comprises of phase estimation on the operator G (Eq. (23)) (refer Fig. 8 for an explicit construction), which returns a state that must undergo further arithmetics (realised using quantum arithmetics) before it has a digital representation of F_j stored in the *Fidelity* register. We represent these circuits as QA. Finally, uncomputing every register except the Index and Fidelity registers, we have the required state.

where we have defined the combined state of all registers except index register after estimation to be

$$|\Psi_{j\text{AE}}\rangle_{\text{ph,tr,tst},B} = \frac{-i}{\sqrt{2}} \left(e^{i\pi\theta_j} |\theta_j\rangle_{\text{ph}} |\Psi_{j+}\rangle_{\text{tr,tst},B} - e^{-i\pi\theta_j} |1-\theta_j\rangle_{\text{ph}} |\Psi_{j-}\rangle_{\text{tr,tst},B} \right). \quad (29)$$

Here, $|\theta_j\rangle_{\text{ph}}$ and $|1-\theta_j\rangle_{\text{ph}}$ are b -qubit states storing b -bit binary representation of θ_j and $1-\theta_j$ respectively.

8. Introducing a separate register, named *fid*, compute $F_j = 2\sin^2(\pi\theta_j) - 1$ using quantum arithmetic (Appendix C). Note that $\sin(\pi\theta_j) = \sin(\pi(1-\theta_j))$, and F_j is uniquely recovered. Then our total state is

$$\xrightarrow{\text{q. arithmetics}} |j\rangle_{\text{in}} |F_j\rangle_{\text{fid}} |\Psi_{j,\text{AE}}\rangle_{\text{ph,tr,tst},B}. \quad (30)$$

9. Uncompute everything in registers phase, train, test and B to get

$$\xrightarrow{\text{uncompute ph, tr, tst, } B} |j\rangle_{\text{in}} |F_j\rangle_{\text{fid}}. \quad (31)$$

Now, we have successfully converted the fidelity values from amplitudes to digital format. Therefore steps 5-9 makes up the operator $\mathcal{E}^{\text{dig}}$ given in Eq. (11). 2-9 gives the construction of the gate $\mathcal{F}$ given in Eq. (9). We

now have an operator capable of the transformation $|j\rangle|0\rangle \xrightarrow{\mathcal{F}} |j\rangle|F_j\rangle$ for arbitrary index j .

10. On separate registers, named index' and fidelity', initialised as $|y\rangle_{\text{in}'}|0\rangle_{\text{fid}'}$, apply $\mathcal{F}$, to obtain

$$\xrightarrow{\mathcal{F}} |j\rangle_{\text{in}} |F_j\rangle_{\text{fid}} |y\rangle_{\text{in}'} |F_y\rangle_{\text{fid}'}. \quad (32)$$

11. Add an extra qubit Q_1 and apply the classical comparison gate (refer Appendix J for more details)

$$J|a\rangle|b\rangle|0\rangle = \begin{cases} |a\rangle|b\rangle|1\rangle & : a > b, \\ |a\rangle|b\rangle|0\rangle & : a \leq b, \end{cases} \quad (33)$$

on registers fid and fid' to get the state

$$\xrightarrow{J} |j\rangle_{\text{in}} |F_j\rangle_{\text{fid}} |y\rangle_{\text{in}'} |F_y\rangle_{\text{fid}'} |g(j)\rangle_{Q_1} \quad (34)$$

where

$$g(j) = \begin{cases} 1 & : F_j > F_y, \\ 0 & : F_j \leq F_y, \end{cases} \quad (35)$$

The qubit Q_1 will mark all indices j such that $F_j > F_y$.

12. Uncompute the registers in' and fid' to obtain the state

$$\xrightarrow{\text{uncompute in', fid'}} |j\rangle_{\text{in}} |F_j\rangle_{\text{fid}} |g(j)\rangle_{Q_1}. \quad (36)$$

13. Add an extra qubit Q_2 and for every $i_l \in A$, apply the gate $D^{(i_l)}$ of the form

$$D^{(i_l)}|j\rangle|0\rangle = \begin{cases} |j\rangle|1\rangle & : j = i_l, \\ |j\rangle|0\rangle & : j \neq i_l. \end{cases} \quad (37)$$

on registers index and Q_2 to get the state

$$\xrightarrow{(D^{(i_1)} \dots D^{(i_k)})_{\text{in}, Q_1}} |j\rangle_{\text{in}} |F_j\rangle_{\text{fid}} |g(j)\rangle_{Q_1} |\chi_A(j)\rangle_{Q_2}, \quad (38)$$

where $\chi_A(j) = 1$ if $j \in A$ and 0 otherwise, is the indicator function of the set A . That is, the sequence of operators $D^{(i_1)} \dots D^{(i_k)}$ marks all the indices that are already in the threshold index set A as we would like to avoid these indices so as to not have repetition in our top k neighbors. These gates can be realized using classical gates (refer Appendix J for more details) by preparing the state $|i_l\rangle$ in an additional m -qubit register, which we uncomputed and recycled from step 12, and using comparison gates.

14. Add an extra qubit Q_3 . Then apply an X gate on Q_2 and a Toffoli gate with controls Q_1, Q_2 and target Q_3 . This results in the state

$$\xrightarrow{X, \text{Toffoli}} |j\rangle_{\text{in}} |F_j\rangle_{\text{fid}} |g(j)\rangle_{Q_1} |\chi_A(j)\rangle_{Q_2} |f_{y,A}(j)\rangle_{Q_3} \quad (39)$$

Note that ultimately we are trying to construct an oracle capable of marking indices j which have $g(j) = 1$ as well as $\chi_A(j) = 0$. An X gate combined with a Toffoli gate will flip the target qubit if one of the input qubits is 0 and the other is 1.

15. Uncomputing every register except index and Q_3 , we have

$$\xrightarrow{\text{uncompute}} |j\rangle_{\text{in}} |f_{y,A}(j)\rangle_{Q_3}. \quad (40)$$

Since the construction of this circuit does not depend on j , we now have an operator that does the aforementioned transformation.

$$\mathcal{O}_{y,A}|j\rangle|0\rangle = |j\rangle|f_{y,A}(j)\rangle. \quad (41)$$

This completes the construction of the oracle $\mathcal{O}_{y,A}$ (7). We may now use this oracle in the k maxima finding algorithm to find the k nearest neighbors of a test state based on fidelity.

4 Quantum k Nearest neighbors algorithm using dot product

We briefly discuss how one can construct a QkNN algorithm that utilises dot product $X(u, v) \equiv \langle u|v \rangle$ instead of fidelity. For real-valued vectors, which is usually the case with real-world practical applications, dot product is more useful. Note that for QkNN using fidelity, we are performing Swap test between the test state and all train states in superposition to analog-encode the fidelity information F_j , and we then use abs-QADC algorithm to digitise this information. For QkNN using dot product, we replace Swap test with Hadamard test (refer Appendix D), i.e., we perform Hadamard test between the test state and all the train states in superposition to analog-encode the dot product information X_j , and then use a similar modification to the real-QADC algorithm to digitise this information. Note that even for complex-valued vectors $|\psi\rangle, |\phi\rangle$, real-QADC returns the real part of the inner product $\text{Re}\langle\psi|\phi\rangle$, and thereby can be of use if the situation agrees.

The steps of this procedure are similar to QkNN using fidelity. Therefore we refrain from presenting it here and instead present it in full detail in Appendix E.

5 Complexity analysis

The fidelity based QkNN requires $O(\sqrt{kM})$ calls to the oracle $\mathcal{O}_{y,A}$. The oracle contains two uses of the $\mathcal{F}$ circuit, which is a slight modification of the abs-QADC algorithm. Therefore the complexity of executing the oracle is similar to the abs-QADC algorithm. For a precision parameter $\epsilon = 2^{-b}$, the circuit of the oracle contains $O(1/\epsilon)$ controlled $\mathcal{V}$ and $\mathcal{W}$ gates and $O((\log^2 N)/\epsilon)$ single and 2 qubit gates. So, one can execute the whole fidelity based QkNN procedure with $O(\sqrt{kM}/\epsilon)$ calls to the data preparation oracles $\mathcal{V}$ and $\mathcal{W}$ and $O(\sqrt{kM}(\log^2 N)/\epsilon)$ single and 2 qubit gates, $O(k\sqrt{kM})$ $D^{(i_l)}$ gates, each of which can be realized in $O(\log(M))$ with an extra register of $\log M$ qubits and $O(\sqrt{kM})$ J gates, each of which can be realized in $O(b)$.

When $N \gg 1/\epsilon$, QkNN algorithm will be a better choice than other k NN algorithms that require the description of the test state since reading the description of the state will have com-

plexity $O(N)$, regardless of how their complexity is related to ϵ . Another approach that can bypass the process of reading the description of the state is by estimating the fidelity through sampling from measurements. For example, one can carry out swap test between the test state and each train state separately, estimate fidelity in each case, and then find the k nearest neighbours. Now, in this method, since the fidelity values are estimated by sampling, to achieve an error bound of ϵ , the test state preparation circuit $\mathcal{V}$ has to be executed $O(1/\epsilon^2)$ times. Then, the method will have a query complexity which is at least $O(M/\epsilon^2)$. The proposed QkNN has a query complexity of $O(\sqrt{kM}/\epsilon)$. So, in this scenario also, QkNN has a potential advantage.

The QkNN algorithm can be executed in $O(\log M) + O(\log N) + O(\log 1/\epsilon)$ qubits. To perform $\mathcal{F}$ once, we need $\log M + b$ qubits and another $2\log N + b + 1$ qubits which are uncomputed at step 9. Considering the fact that the uncomputed qubits can be recycled, applying $\mathcal{F}$ on two separate registers will require $2(\log M + b) + 2\log N + b + 1$ qubits. The classical operations would require 3 qubits as well as $O(\log M)$ qubits for the $D^{(i)}$ gates and $O(b)$ qubits for the J gates. Let k_J be the number of qubits required for the J gates and let k_D be the number of qubits required for the $D^{(i)}$ gates. Since the $2\log N + b + 1$ qubits can be recycled and after using the J gates, the states prepared in registers in', fid' can be uncomputed, to execute the QkNN algorithm using fidelity measure, we require $2\log M + 2b + k_J + 1 + \max(k_D + 2 - \log M - b, 2\log N + b - k_J)$. For example, to carry out QkNN for 30 qubit states (dimension ≈ 1 billion) with 2^{10} train states, we would require around $84 + 3b$ qubits, along with any ancilliary qubits required k_D and k_J .

The dot product based QkNN has the same query complexity as the fidelity based QkNN. To perform this operation, one would require $2\log M + 2b + k_J + 1 + \max(k_D + 2 - \log M - b, \log N + b - k_J)$ qubits.

6 Applications

We present two scenarios where our quantum kNN algorithm can be applied. In both scenarios, we utilize the fact that we can classify an unknown test state.

6.1 Entanglement classification

6.1.1 Entanglement classes

In this section, we discuss the entanglement classes in pure n -partite quantum states. For simplicity, we restrict ourselves to n -qubit systems. We begin with $n = 2$ case. A pure two-qubit quantum state $|\Phi\rangle$ is called separable or product state if and only if it can be written as a tensor product of two pure states corresponding to individual subsystems,

$$|\Phi\rangle = |\phi_1\rangle \otimes |\phi_2\rangle. \quad (42)$$

If the state $|\Phi\rangle$ is not of the form (42) then its an entangled state. A pure n -qubit quantum state $|\Psi\rangle$ is separable only if it can be written as the tensor product of n quantum states as

$$|\Psi\rangle = |\psi_1\rangle \otimes \cdots \otimes |\psi_n\rangle. \quad (43)$$

Such states are also called n -separable states [32]. Equivalently, a pure state $|\Psi\rangle$ is an n -separable state if it is separable across all the possible bipartitions of the n qubits. If this condition is violated, then the state is no longer n -separable. Some states can be entangled in certain bipartitions and separable in others. Some states are entangled across all bipartitions. This motivates a classification of n -partite quantum states on the basis of entanglement. For two-qubit systems, there are only two classes – separable and entangled states.

For three-qubit systems, we have more. Let A, B, and C represent the three qubits, and let us use A-B to denote that subsystems A and B are separable and AB to denote that subsystems A and B are entangled. Then the entanglement classes can be written as $\{A-B-C, AB-C, A-BC, AC-B, ABC\}$. Note that we do not distinguish between W states and GHZ states defined in [33] and keep them in the same class ABC.

The same classification of the entanglement can be extended to n number of qubits. The question that is relevant to us is the following: given a circuit that prepares an n -qubit arbitrary state, is there a way to label it according to its entanglement class. We show that the classical kNN algorithm can classify these states for $n = 2, 3$ with high accuracy. Furthermore, similar accuracy can be achieved by our QkNN algorithm without the classical description of the given quantum state, establishing the advantage of QkNN over

the other k NN algorithms in settings where the input is a circuit which prepares the test state, rather than the description of the state itself.

6.1.2 Simulation results

We present the results of simulation of estimating the entanglement class of a test state $|\psi\rangle$ using classical k NN in Table 1[†]. We use this numerical experiment to demonstrate that quantum states in Hilbert space have a nearest-neighbor structure when it comes to entanglement. That is, the closer the states are, the similar their entanglement is.

For two-qubit states, we demonstrate this for both separable (entanglement entropy = 0) vs entangled (entanglement entropy $\neq 0$) and separable vs maximally entangled (entanglement entropy = 1). We also demonstrate it for three-qubit states with a classification among the five different classes mentioned above.

6.2 Quantum state discrimination

Another application that we propose for Qk NN is a problem analogous to quantum state discrimination [34]. The problem of quantum state discrimination is originally formulated for arbitrary (mixed) states and in terms of measurement. Consider a collection of states $\mathcal{S} = \{\rho_0, \dots, \rho_{M-1}\}$ with associated probabilities $p = (p_0, \dots, p_{M-1})$. A state is drawn from $\mathcal{S}$ according to p and is prepared. The aim is to find a measurement which maximizes the probability of correctly identifying the prepared state.

We are concerned with an analogous problem. Suppose we are given a circuit that produces an unknown (pure) state $|\psi\rangle$. We are guaranteed that $|\psi\rangle$ is one of the M known (pure) states $\{|\phi_0\rangle, \dots, |\phi_{M-1}\rangle\}$. We also assume that we are given circuit $\mathcal{W}$ to prepare the known states. The task is to correctly deduce the identity of $|\psi\rangle$.

In such a scenario, one can run the Qk NN algorithm with $k = 1$ to obtain the j such that $|\psi\rangle = |\phi_j\rangle$ with $O(\sqrt{M})$ oracle calls, as the largest value fidelity can take is 1 and $F(u, v) = 1$ if and only if $u = v$.

[†]The simulation code can be found at <https://github.com/afraadnyf/QKNN>.

7 Conclusion

In this paper, we have presented a novel Qk NN algorithm, which is a quantum analog of the classical k NN algorithm. We use Swap test and generalizations of quantum analog to digital conversion algorithms to construct an oracle which enables us to reduce the problem of quantum k NN to an instance of quantum k maxima finding algorithm. We assume that state preparations circuits are provided, and the algorithm uses fidelity as a similarity measure which is widely used in problems where the data is inherently quantum. The algorithm requires $O(\sqrt{kM})$ calls to these circuits to obtain the identities of the k nearest neighbors of the test out of M train states. Since the metric Bures distance is a monotonous function of fidelity, k NN done using fidelity is in agreement with k NN carried out using Bures distance. We also present a variant of the algorithm which uses dot product as distance measure.

An advantage of the proposed algorithm is its ability to classify quantum states without their explicit classical description in some basis. Instead, we require circuits capable of preparing these states. Furthermore, while dealing with quantum data, the algorithm is able to classify without the requirement of quantum state tomography, which is essential if one were to use any other, quantum or classical, k NN algorithm which requires the classical description of states. Along with the calls to the state preparation circuits being $O(\sqrt{kM})$, all the other parts of the circuit are efficiently preparable. The number of qubits required is also poly-logarithmic in the dimension of the states involved and the number of train states.

As an example, we show the effectiveness of k NN method in identifying the type of entanglement in quantum states. For the problem of entanglement classification, preparing train states of different types of entanglement is much easier when we're working with their circuits rather than working with their classical descriptions. Furthermore, we discuss the applicability of the algorithm in an analogous version of quantum state discrimination.

A particular future direction regarding the applications of Qk NN is to study its capability in an analogous version of quantum gate discrimination [35]. This is due to the ability of Qk NN to work with quantum circuits which could be

No. of Qubits	No. of classes	Entanglement classes	Accuracy
2	2	Separable, Entangled	99%
2	2	Separable, Maximally entangled	100%
3	5	1-2-3, 12-3, 1-23, 13-2, 123	89%

Table 1: Entanglement classification using classical k NN classifier. Cardinality of the set of train states is $M = (\text{number of classes}) \times (\text{class size})$. In each case, the total number of train states used for each class is 10^5 .

efficient representations of unitary matrices. The problem of identifying a test circuit among a finite set of train circuits can potentially be addressed using a column-wise Q k NN.

8 Acknowledgements

We thank Yuan Feng, Sanjiang Li, and Christopher Ferrie for fruitful discussions. SKG acknowledges the financial support from SERB-DST (File No. ECR/2017/002404) and Interdisciplinary Cyber Physical Systems(ICPS) programme of the Department of Science and Technology, India (Grant No.:DST/ICPS/QuST/Theme-1/2019/12). The quantum circuits were generated using the Quantikz package [36] and Mathcha [37].

References

- [1] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, [Quantum machine learning - *Nature* **549**, 195–202 \(2017\).](#)
- [2] P. Wittek, [Quantum machine learning - *What quantum computing means to data mining* \(2017\).](#)
- [3] M. Schuld, I. Sinayskiy, and F. Petruccione,, [An introduction to quantum machine learning \(2014\).](#)
- [4] S. Arunachalam and R. de Wolf , [A Survey of Quantum Learning Theory - *arXiv:1701.06806* \(2017\).](#)
- [5] A. Harrow, A. Hassidim, and S. Lloyd, [Quantum Algorithm for Linear Systems of Equations - *Phys. Rev. Lett.* **103**, 150502 \(2009\).](#)
- [6] N. Wiebe, D. Braun, and Seth Lloyd, [Quantum Algorithm for Data Fitting - *Phys. Rev. Lett.* **109**, 050505 \(2012\).](#)
- [7] J. Carrasquilla and R. Melko, [Machine learning phases of matter - *Nature Phys* **13**, 431–434 \(2017\).](#)
- [8] B. Wang, [Learning to Detect Entanglement - *arXiv:1709.03617* \(2017\).](#)
- [9] S. Lu, S. Huang, K. Li, J. Li, Jianxin Chen, Dawei Lu, Zhengfeng Ji, Yi Shen, Duanlu Zhou, and Bei Zeng, [Separability-entanglement classifier via machine learning - *Phys. Rev. A* **98**, 012315 \(2018\).](#)
- [10] G. Carleo, I. Cirac, K. Cranmer, L. Daudet, M. Schuld, N. Tishby, L. Vogt-Maranto, and L. Zdeborová, [Machine learning and the physical sciences - *Rev. Mod. Phys.* **91**, 045002 \(2019\).](#)
- [11] S. Lloyd, M. Mohseni, P. Rebentrost, [Quantum principal component analysis - *Nature Physics*, **10**, 631–633 \(2014\).](#)
- [12] P. Rebentrost, M. Mohseni, and Seth Lloyd, [Quantum Support Vector Machine for Big Data Classification - *Phys. Rev. Lett.* **113**, 130503 \(2014\).](#)
- [13] S. Lloyd, M. Mohseni, and P. Rebentrost, [Quantum algorithms for supervised and unsupervised machine learning - *arXiv:1307.0411* \(2013\).](#)
- [14] T. Cover and P. Hart, [Nearest neighbor pattern classification - *IEEE Transactions on Information Theory*, **13**, 1, 21-27 \(1967\).](#)
- [15] C. Dürr, M. Heiligman, P. Hoyer, and M. Mhalla, [Quantum Query Complexity of Some Graph Problems, *SIAM Journal on Computing* **35**, 1310-1328 \(2006\).](#)
- [16] K. Miyamoto, M. Iwamura, K. Kise, [A Quantum Algorithm for Finding k-Minima - *arXiv:1907.03315* \(2019\).](#)
- [17] H. Buhrman, R. Cleve, J. Watrous, and R. de Wolf, [Quantum Fingerprinting - *Phys. Rev. Lett.* **87**, 167902 \(2001\).](#)

- [18] K. Mitarai, M. Kitagawa, and K. Fujii, Quantum analog-digital conversion - *Phys. Rev. A* **99**, 012301 (2019).
- [19] S. Aaronson, The learnability of quantum states - *Proc. R. Soc. A.* 4633089–3114 (2007).
- [20] N. Wiebe, A. Kapoor, and K. M. Svore, Quantum algorithms for nearest-neighbor methods for supervised and unsupervised learning - *Quantum Information & Computation*, **15**, 3-4, 316–356 (2015).
- [21] Y. Ruan, x. Xue, H. Liu, J. Tan, and X. Li, Quantum Algorithm for K-Nearest Neighbors Classification Based on the Metric of Hamming Distance - *International Journal of Theoretical Physics*, **56**, 3496–3507 (2017).
- [22] H. Chen, Y. Gao, and J. Zhang, Quantum K-nearest neighbor algorithm - *Dongnan Daxue Xuebao* **45**, 647 (2015).
- [23] Y. Dang, N. Jiang, H. Hu, Z. Ji, and W. Zhang, Image classification based on quantum K-Nearest-Neighbor algorithm - *Quantum Information Processing*, **17**, 239 (2018).
- [24] M. Schuld, I. Sinayskiy, and F. Petruccione, Quantum Computing for Pattern Classification - *PRICAI 2014: Trends in Artificial Intelligence, Lecture Notes in Computer Science*, 8862, Springer (2014).
- [25] Y. Liao and V. R. Vemuri, Use of K-Nearest Neighbor classifier for intrusion detection - *Computers & Security*, **21**, 5, 439–448 (2002).
- [26] J.P. Zhang and I. Mani, KNN Approach to Unbalanced Data Distributions: A Case Study Involving Information Extraction - *Proceeding of International Conference on Machine Learning, Workshop on Learning from Imbalanced Data Sets, Washington DC*, 21 (2003).
- [27] R. J. Samworth, Optimal weighted nearest neighbour classifiers - *The Annals of Statistics*, **40**, 5, 2733–2763 (2012).
- [28] C. Durr and P. Hoyer, A Quantum Algorithm for Finding the Minimum - *quant-ph/9607014* (1996).
- [29] M. Boyer, G. Brassard, P. Hoyer, and A. Tappa, Tight Bounds on Quantum Searching - *Fortschritte der Physik*, **46**, 187–199 (2005).
- [30] M. A. Nielsen and I. L. Chuang, Quantum Computation and Quantum Information: 10th Anniversary Edition - *Cambridge University Press* (2011).
- [31] Iordanis Kerenidis, Anupam Prakash, Quantum Recommendation Systems - *arXiv:1603.08675* (2016).
- [32] R. Horodecki, P. Horodecki, M. Horodecki, and K. Horodecki, Quantum entanglement - *Rev. Mod. Phys.* **81**, 865 (2009).
- [33] W. Dür, G. Vidal, and J. I. Cirac, Three qubits can be entangled in two inequivalent ways - *Phys. Rev. A* **62**, 062314 (2000).
- [34] J. Bae and L. Kwek, Quantum state discrimination and its applications - *Journal of Physics A: Mathematical and Theoretical*, **48**, 8 (2015).
- [35] G. Chiribella, G. M. D’Ariano, and M. Roetteler, Identification of a reversible quantum gate: Assessing the resources - *New Journal of Physics*, **15** (2013).
- [36] A. Kay, Tutorial on the Quantikz Package - *arXiv:1809.03842* (2018).
- [37] Mathcha, Mathcha - online mathematics editor, (2021).
- [38] R. Cleve, A. Ekert, C. Macchiavello and M. Mosca, Quantum algorithms revisited - *Proc. R. Soc. Lond. A* 454339–354 (1998).
- [39] L. Ruiz-Perez and J. C. Garcia-Escartin, Quantum arithmetic with the quantum Fourier transform - *Quantum Information Processing*, **16**, 152 (2017).
- [40] D. S. Oliveira, P. B. M. de Sousa and R. V. Ramos, Quantum search algorithm using quantum bit string comparator - *2006 International Telecommunications Symposium*, 582–585 (2006).

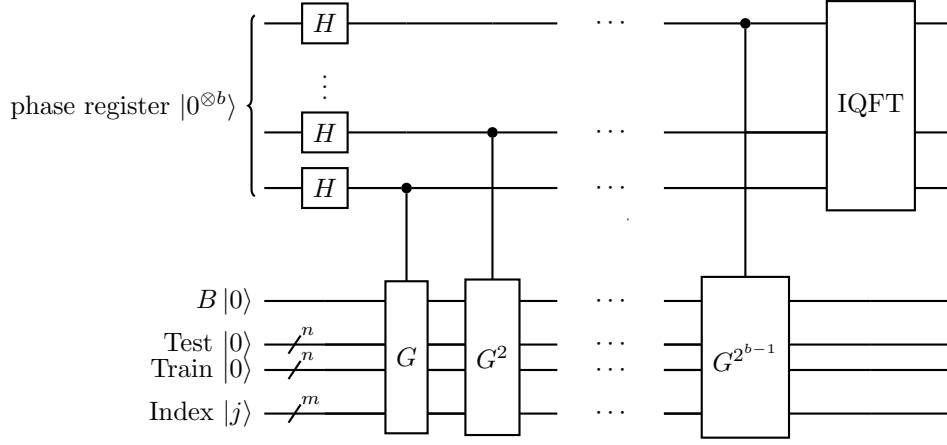


Figure 8: Quantum phase estimation on an operator G . This is the innards of the ‘PhaseEst on G ’ operator in Fig. 7.

A Quantum Analog-Digital Conversion algorithms

Mitarai et al. describes a set of algorithms [18] to carry out analog-digital conversions within a quantum circuit. Let $U|0\rangle = \sum_{i=0}^{d-1} c_i|i\rangle$ be an arbitrary quantum state.

Definition 1. abs-QADC [18] Let $\tilde{r}_j$ denote the m -bit string $\tilde{r}_j^1, \tilde{r}_j^2, \dots, \tilde{r}_j^m$ that best approximates $|c_j|$ by $\sum_{k=1}^m \tilde{r}_j^k 2^{-k}$. An m -bit abs-QADC operation transforms analog-encoded state $\sum_{j=1}^N c_j|j\rangle|0\rangle^{\otimes m}$ to $\frac{1}{\sqrt{N}} \sum_{j=1}^N |j\rangle|\tilde{r}_j\rangle$.

Definition 2. real-QADC [18] Let $\tilde{x}_j$ denote the m -bit string $\tilde{x}_j^1, \tilde{x}_j^2, \dots, \tilde{x}_j^m$ that best approximates the real part of c_j by $\sum_{k=1}^m \tilde{x}_j^k 2^{-k}$. An m -bit real-QADC operation transforms analog-encoded state $\sum_{j=1}^N c_j|j\rangle|0\rangle^{\otimes m}$ to $\frac{1}{\sqrt{N}} \sum_{j=1}^N |j\rangle|\tilde{x}_j\rangle$.

Theorem 1. abs-QADC [18] There exists an m -bit abs-QADC algorithm that runs using $O(1/\epsilon)$ controlled- U gates and $O((\log^2 N)/\epsilon)$ single and two qubit gates with output state fidelity $(1 - O(\text{poly}(\epsilon)))$, where $\epsilon = 2^{-m}$.

Theorem 2. real-QADC [18] There exists an m -bit real-QADC algorithm that runs using $O(1/\epsilon)$ controlled- U gates and $O((\log^2 N)/\epsilon)$ single and two qubit gates with output state fidelity $(1 - O(\text{poly}(\epsilon)))$, where $\epsilon = 2^{-m}$.

B Quantum Phase Estimation

Quantum phase estimation is a quantum procedure that can be used to estimate the phase of the eigenvalue of a given eigenvector of a unitary operator. It relies on the quantum Fourier transform and is the engine behind some of the most popular quantum algorithms such as the Shor’s algorithm for factoring.

Theorem 3. [38] Let U be a unitary operator acting on M -qubit Hilbert space with eigenstates $\{|\psi_j\rangle\}_{j=1}^{2^M}$ and corresponding eigenvalues $\{e^{2\pi i \phi_j}\}_{j=1}^{2^M}$ where $\phi_j \in [0, 1)$. Let $\epsilon = 2^{-m}$ for some positive integer m . There exists a quantum algorithm, which consists of $O(1/\epsilon)$ controlled- U calls and

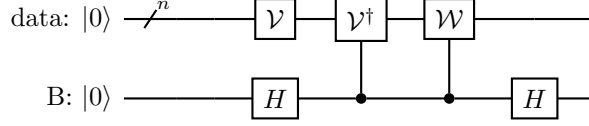


Figure 9: Circuit for Hadamard test.

$O(\log^2(1/\epsilon))$ single and two-qubit gates, that performs transformation $\sum_{j=1}^{2^M} a_j |\psi_j\rangle |0\rangle^{\otimes m} \rightarrow |\psi_{PE}\rangle = \sum_{j=1}^{2^M} a_j |\psi_j\rangle |\tilde{\phi}_J\rangle$ where $|\tilde{\phi}_J\rangle$ denotes a bitstring $\tilde{\phi}_J^{(1)} \tilde{\phi}_J^{(2)} \dots \tilde{\phi}_J^{(m)}$ such that $\left| \sum_{k=1}^m \tilde{\phi}_J^{(k)} 2^{-k} - \phi_J \right| \leq \epsilon$ for all j with state fidelity at least $1 - \text{poly}(\epsilon)$.

C Quantum arithmetics

Within a quantum circuit, one can always apply simple arithmetic functions such as additions, multiplication, trigonometric functions, exponentiation, etc. This is explained using the following theorems.

Theorem 4. [39] Let $\mathbf{a}, \mathbf{b}$ be m -bit strings. There exists a quantum algorithm that performs transformation $|\mathbf{a}\rangle|\mathbf{b}\rangle \rightarrow |\mathbf{a}\rangle|\mathbf{a} + \mathbf{b}\rangle$ with $O(\text{poly}(m))$ single and two qubit gates.

Using this quantum adder, one can construct a circuit capable of carrying out any basic function in a similar manner in a quantum circuit.

Theorem 5. [18] Some basic functions such as inverse, trigonometric functions, square root, and inverse trigonometric functions can be calculated to accuracy ϵ , that is, we can perform a transformation $|a\rangle|0\rangle \rightarrow |a\rangle|\mathbf{f}(\mathbf{a})\rangle$ such that $|\mathbf{f}(\mathbf{a}) - \mathbf{f}(\mathbf{a})| \leq \epsilon$ where f is the required function using $O(\text{poly}(\log(1/\epsilon)))$ quantum arithmetics.

D Hadamard Test

The Hadamard test is a quantum circuit that can be used to compute the real part or the imaginary part of the inner product between two quantum states $|u\rangle, |v\rangle \in \mathbb{C}^n$. Since, we are more interested in the real part of the inner product, we will be using and explaining that particular version of the Hadamard test.

Let $U|0\rangle = |u\rangle$ and $V|0\rangle = |v\rangle$. The aim is to use U and V in a controlled manner to construct the state

$$\frac{1}{2} \left[|0\rangle (|u\rangle + |v\rangle) + |1\rangle (|u\rangle - |v\rangle) \right]. \quad (44)$$

The detailed circuit is shown in Fig. 9. The real part of the inner product can be estimated from measuring the first qubit as

$$\Pr(0) = \frac{1}{2} + \frac{1}{2} \text{Re}(\langle u|v\rangle) \quad \text{and} \quad \Pr(1) = \frac{1}{2} - \frac{1}{2} \text{Re}(\langle u|v\rangle). \quad (45)$$

The quantity $\Pr(0) - \Pr(1)$ gives us the desired real part of the inner product.

E Quantum k Nearest neighbors algorithm using dot product

A more general distance measure used in k NN problems is the dot product. Let $\mathcal{H}$ be the n -qubit Hilbert space and let $|v\rangle \in \mathcal{H}$ be the real valued test state whose label is to be determined. Let $\{|u_i\rangle : i \in \{0, \dots, M-1\}\} \subset \mathcal{H}$ be a collection of M known real valued train states whose labels are

known to us. Let $X_i \equiv X(v, u_i) = \langle v | u_i \rangle$ be the dot product between the test state $|v\rangle$ and the i^{th} train state $|u_i\rangle$ and

$$X = [X_0, \dots, X_{M-1}] \quad (46)$$

be a table of length M containing the dot product values with the test state $|v\rangle$ and all the train states $\{|u_i\rangle\}$. The general approach adopted here is very similar to fidelity based QkNN. One can see that the problem of QkNN using dot product is an instance of the quantum k maxima finding algorithm on the set X given in Eq. (46). Similar to QkNN using fidelity, the problem boils down to being able to perform quantum search on the Boolean function

$$f_{y,A}(j) = \begin{cases} 1 & : X_j > X_y \text{ and } j \notin A, \\ 0 & : \text{otherwise.} \end{cases} \quad (47)$$

That is, we should be able to prepare the oracle

$$\mathcal{O}_{y,A}|j\rangle|0\rangle = \begin{cases} |j\rangle|1\rangle & : X_j > X_y \text{ and } j \notin A, \\ |j\rangle|0\rangle & : \text{otherwise.} \end{cases} \quad (48)$$

The assumptions here are that we are provided with *state preparation* oracles $\mathcal{V}, \mathcal{W}$ of the form

$$|0^n\rangle \xrightarrow{\mathcal{V}} |v\rangle, \quad (49)$$

$$|j\rangle|0^n\rangle \xrightarrow{\mathcal{W}} |j\rangle|u_j\rangle, \quad (50)$$

for $j \in \{0, \dots, M-1\}$. The construction of the oracle is based on the real-QADC circuit from [18]. In that circuit, to compute the real values of the coordinates of the state, we apply the Hadamard test in superposition with the state and standard basis vectors. In the quantum kNN setting, we apply the Hadamard test in superposition with the test state and the train states. We now show the correctness of such a protocol and a method to build the required oracle $\mathcal{O}_y$ using it. The explicit construction of the oracle is as follows:

1. Initialise three registers named *index*, *data*, *B* of sizes $m, n, 1$ respectively, where $n = \log N$ and $m = \log M$

$$|j\rangle_{\text{in}}|0^n\rangle_{\text{data}}|0\rangle_B. \quad (51)$$

2. Apply $\mathcal{V}$ on the data register

$$\xrightarrow{\mathcal{V}} |j\rangle_{\text{in}}|v\rangle_{\text{data}}|0\rangle_B. \quad (52)$$

3. Perform Hadamrd test to obtain the state

$$\begin{aligned} & \xrightarrow{\text{Hadamard Test}} \frac{1}{2} |j\rangle_{\text{in}} \left[\left(|v\rangle_{\text{data}} + |u_j\rangle_{\text{data}} \right) |0\rangle_B + \left(|v\rangle_{\text{data}} - |u_j\rangle_{\text{data}} \right) |1\rangle_B \right] \\ & \equiv |j\rangle_{\text{in}} |\Psi_j\rangle_{\text{data},B}, \end{aligned} \quad (53)$$

where,

$$|\Psi_j\rangle_{\text{data},B} = \frac{1}{2} \left[\left(|v\rangle_{\text{data}} + |u_j\rangle_{\text{data}} \right) |0\rangle_B + \left(|v\rangle_{\text{data}} - |u_j\rangle_{\text{data}} \right) |1\rangle_B \right]. \quad (54)$$

Define V to be the combined unitary transformations of steps 2 and 3. If one now measures the B register, one would see the probabilities as

$$\Pr(B=0) = \frac{1+X_j}{2} \quad \text{and} \quad \Pr(B=1) = \frac{1-X_j}{2}. \quad (55)$$

The information regarding dot product is now encoded in the amplitudes. We must now convert it into a ‘digital’ format which can be further utilised.

4. Construct a gate

$$H = V_{\text{in,data},B} S_{0,\text{data},B} V_{\text{in,data},B}^\dagger Z_B, \quad (56)$$

where $S_0 = \mathbb{1} - 2|0\rangle\langle 0|$. This H gate can be seen as the H gate in the real QADC algorithm [18], with the controlled CNOT gate replaced by controlled $\mathcal{W}$. The action of H on the current state can be written as controlled action of operators H_j (refer Appendix H):

$$H|j\rangle_{\text{in}}|\Psi_j\rangle_{\text{data},B} = |j\rangle_{\text{in}}(H_j|\Psi_j\rangle_{\text{data},B}), \quad (57)$$

where

$$H_j = (\mathbb{1} - 2|\Psi_j\rangle\langle\Psi_j|_{\text{data},B}) Z_B. \quad (58)$$

5. $|\Psi_j\rangle_{\text{tr,tst},B}$ can be decomposed into two eigenstates of H_j , namely $|\Psi_{j+}\rangle$ and $|\Psi_{j-}\rangle$, corresponding to the eigenvalues $e^{\pm i2\pi\theta_j}$, respectively. Here, $\sin(\pi\theta_j) = \sqrt{\frac{1}{2}(1+X_j)}$ and $\theta \in [1/4, 1/2)$ (refer Appendix I). The decomposition is given as

$$|\Psi_j\rangle = \frac{-i}{\sqrt{2}}(e^{i\pi\theta_j}|\Psi_{j+}\rangle - e^{-i\pi\theta_j}|\Psi_{j-}\rangle), \quad (59)$$

6. The operator H has the dot product values $\{X_j\}$ stored in its eigenvalues. To get a b -bit binary representation of θ_j , we now run the phase estimation algorithm on H . To this end, we bring the *phase register* containing b qubits and run the phase estimation algorithm:

$$\begin{aligned} & \xrightarrow{\text{PhaseEst.}} \frac{-i}{\sqrt{2}}|j\rangle_{\text{in}}[e^{i\pi\theta_j}|\theta_j\rangle_{\text{ph}}|\Psi_{j+}\rangle_{\text{data},B} - e^{-i\pi\theta_j}|1-\theta_j\rangle_{\text{ph}}|\Psi_{j-}\rangle_{\text{data},B}] \\ & \equiv |j\rangle_{\text{in}}|\Psi_{j,\text{AE}}\rangle_{\text{ph,data},B}, \end{aligned} \quad (60)$$

where we have defined the combined state of all registers except index register after estimation to be

$$|\Psi_{j,\text{AE}}\rangle_{\text{ph,data},B} = \frac{-i}{\sqrt{2}}(e^{i\pi\theta_j}|\theta_j\rangle_{\text{ph}}|\Psi_{j+}\rangle_{\text{data},B} - e^{-i\pi\theta_j}|1-\theta_j\rangle_{\text{ph}}|\Psi_{j-}\rangle_{\text{data},B}). \quad (61)$$

Here, $|\theta_j\rangle_{\text{ph}}$ and $|1-\theta_j\rangle_{\text{ph}}$ are b -qubit states storing b -bit binary representation of θ_j and $1-\theta_j$ respectively.

7. Introducing a register, dp , compute $X_j = 2\sin^2(\pi\theta_j) - 1$ using quantum arithmetic from Appendix C. Note that $\sin(\pi\theta_j) = \sin(\pi(1-\theta_j))$, and X_j is uniquely recovered. Then our total state is

$$\xrightarrow{\text{quantum arithmetics}} |j\rangle_{\text{in}}|X_j\rangle_{\text{dp}}|\Psi_{j,\text{AE}}\rangle_{\text{ph,data},B}. \quad (62)$$

8. Uncompute everything in registers ph,data and B to get

$$\xrightarrow{\text{uncompute ph, data, B}} |j\rangle_{\text{in}}|X_j\rangle_{\text{dp}}. \quad (63)$$

We have successfully converted the dot product values from amplitudes to digital format.

9. Add more registers and apply $\mathcal{X}$, so that we have

$$\xrightarrow{\mathcal{X}} |j\rangle_{\text{in}}|X_j\rangle_{\text{dp}}|y\rangle_{\text{in}'}|X_y\rangle_{\text{dp}'} \quad (64)$$

10. Add an extra qubit Q_1 and apply the gate J defined in Eq. (11) on registers dp and dp' to get the state

$$\xrightarrow{J} |j\rangle_{\text{in}}|X_j\rangle_{\text{dp}}|y\rangle_{\text{in}'}|X_y\rangle_{\text{dp}'}|g(j)\rangle_{Q_1} \quad (65)$$

where

$$g(j) = \begin{cases} 1 & : X_j > X_y, \\ 0 & : X_j \leq X_y, \end{cases} \quad (66)$$

11. Uncompute the registers in' and dp' to obtain the state

$$\xrightarrow{\text{uncompute in}', \text{dp}'} |j\rangle_{\text{in}} |X_j\rangle_{\text{dp}} |g(j)\rangle_{Q_1}. \quad (67)$$

12. Add an extra qubit Q_2 and for every $i_l \in A$, apply the gate $D^{(i_l)}$ defined in Eq. (13) on registers index and Q_2 to get the state

$$\xrightarrow{(D^{(i_1)} \dots D^{(i_k)})_{\text{in}, Q_1}} |j\rangle_{\text{in}} |X_j\rangle_{\text{dp}} |g(j)\rangle_{Q_1} |\chi_A(j)\rangle_{Q_2}, \quad (68)$$

where $\chi_A(j) = 1$ if $j \in A$ and 0 otherwise, is the indicator function of the set A .

13. Add an extra qubit Q_3 . Then apply an X gate on Q_2 and a Toffoli gate with controls Q_1, Q_2 and target Q_3 . This results in the state

$$\xrightarrow{X, \text{Toffoli}} |j\rangle_{\text{in}} |X_j\rangle_{\text{dp}} |g(j)\rangle_{Q_1} |\chi_A(j)\rangle_{Q_2} |f_{y,A}(j)\rangle_{Q_3}, \quad (69)$$

where $f_{y,A}$ is the Boolean function defined in Eq. (E).

14. Uncomputing every register except index and Q_3 , we have

$$\xrightarrow{\text{uncompute}} |j\rangle_{\text{in}} |f_{y,A}(j)\rangle_{Q_3}, \quad (70)$$

which is the required oracle

$$\mathcal{O}_{y,A} |j\rangle |0\rangle = |j\rangle |f_{y,A}(j)\rangle. \quad (71)$$

This completes the construction of the oracle given in (48). We may now use this oracle in the k maxima finding algorithm to find the k nearest neighbors of a test state based on dot product.

F Action of G as controlled G_j for the Fidelity based Q k NN

Recall that

$$G = U_{\text{tr}, \text{tst}, B} \mathcal{W}_{\text{in}, \text{tr}} S_0 U_{\text{tr}, \text{tst}, B}^\dagger \mathcal{W}_{\text{in}, \text{tr}}^\dagger U_{\text{tr}, \text{tst}, B}^\dagger Z_B, \quad (72)$$

$$G_j = U_{\text{tr}, \text{tst}, B} S_j U_{\text{tr}, \text{tst}, B}^\dagger \quad (73)$$

where

$$S_0 = \mathbb{1} - 2|0\rangle\langle 0|_{\text{tst}, \text{tr}, B}, \quad (74)$$

$$S_j = \mathbb{1} - 2|0\rangle\langle 0|_{\text{tst}, B} \otimes |\phi_k\rangle\langle \phi_k|. \quad (75)$$

To show the equivalence between G and controlled G_k , it suffices to show the equivalence between $\mathcal{W} S_0 \mathcal{W}^\dagger$ and S_k . Recall that we introduced $\mathcal{W}_{\text{ind}, \text{tr}}$ for the preparation of train states. The action of $\mathcal{W}$ is given by

$$\mathcal{W}_{\text{ind}, \text{tr}} |i\rangle_{\text{ind}} |0\rangle_{\text{tr}} = |i\rangle_{\text{ind}} |\phi_i\rangle_{\text{tr}}. \quad (76)$$

We have

$$\mathbb{1}_{\text{in}} \otimes S_0 = \mathbb{1} - 2 \sum_{k=0}^{M-1} |k, 0, 0, 0\rangle\langle k, 0, 0, 0|_{\text{in}, \text{tst}, \text{tr}, B}, \quad (77)$$

and therefore

$$\begin{aligned}
\mathcal{W}S_0\mathcal{W}^\dagger &= \mathcal{W}_{\text{ind, tr}} \left(\mathbb{1} - 2 \sum_{k=0}^{M-1} |k, 0, 0, 0\rangle \langle k, 0, 0, 0|_{\text{in, tst, tr, B}} \right) \mathcal{W}_{\text{ind, tr}}^\dagger \\
&= \mathbb{1} - 2 \sum_{k=0}^{M-1} \mathcal{W} |k, 0, 0, 0\rangle \langle k, 0, 0, 0| \mathcal{W}^\dagger \\
&= \mathbb{1} - 2 \sum_k |k\rangle \langle \phi_k| |0, 0\rangle \langle k| \langle \phi_k| \langle 0, 0| \\
&= \sum_k |k\rangle \langle k|_{\text{in}} \otimes \mathbb{1}_{\text{tst, tr, B}} - 2 \sum_k |k\rangle \langle k|_{\text{in}} \otimes |\phi_k\rangle \langle \phi_k| \otimes |0\rangle \langle 0|_{\text{tst, B}}
\end{aligned} \tag{78}$$

This is

$$\mathcal{W}S_0\mathcal{W}^\dagger = \sum_k |k\rangle \langle k| \otimes \left(\mathbb{1} - 2|0\rangle \langle 0|_{\text{tst, B}} \otimes |\phi_k\rangle \langle \phi_k|_{\text{tr}} \right) = \sum_k |k\rangle \langle k|_{\text{in}} \otimes S_{k\text{tst, tr, B}} \tag{79}$$

as required.

G Eigenvectors of G_j for the fidelity based QkNN

Recall that $G_j = US_jU^\dagger Z_B$ and $U|0\rangle_{\text{tst}}|\phi_{j\text{tr}}\rangle|0\rangle_B = |\Psi_j\rangle$. This implies,

$$US_jU^\dagger = \mathbb{1} - 2|\Psi_j\rangle \langle \Psi_j|. \tag{80}$$

Let

$$|\Psi_{j0}\rangle = \frac{1}{2\alpha_j} \left(|\psi\rangle_{\text{tr}} |\phi_j\rangle_{\text{tst}} + |\phi_j\rangle_{\text{tr}} |\psi\rangle_{\text{tst}} \right) |0\rangle_B \quad \text{and} \quad |\Psi_{j1}\rangle = \frac{1}{2\beta_j} \left(|\psi\rangle_{\text{tr}} |\phi_j\rangle_{\text{tst}} - |\phi_j\rangle_{\text{tr}} |\psi\rangle_{\text{tst}} \right) |1\rangle_B. \tag{81}$$

where,

$$\alpha_j = \sqrt{\frac{1}{2}(1 + F_j)} \quad \text{and} \quad \beta_j = \sqrt{\frac{1}{2}(1 - F_j)}. \tag{82}$$

Therefore we have,

$$|\Psi_j\rangle = \alpha_j |\Psi_{j0}\rangle + \beta_j |\Psi_{j1}\rangle. \tag{83}$$

Consider the subspace $\mathcal{H} = \text{span}(|\Psi_{j0}\rangle, |\Psi_{j1}\rangle)$. We have

$$Z_B|_{\mathcal{H}} = |\Psi_{j0}\rangle \langle \Psi_{j0}| - |\Psi_{j1}\rangle \langle \Psi_{j1}|, \tag{84}$$

and

$$US_jU^\dagger|_{\mathcal{H}} = (1 - 2\alpha_j^2)|\Psi_{j0}\rangle \langle \Psi_{j0}| + (1 - 2\beta_j^2)|\Psi_{j1}\rangle \langle \Psi_{j1}| - 2\alpha_j\beta_j(|\Psi_{j1}\rangle \langle \Psi_{j0}| + |\Psi_{j0}\rangle \langle \Psi_{j1}|). \tag{85}$$

This implies that

$$\begin{aligned}
G_j|_{\mathcal{H}} &= US_jU^\dagger Z_B|_{\mathcal{H}} \\
&= (1 - 2\alpha_j^2)|\Psi_{j0}\rangle \langle \Psi_{j0}| - (1 - 2\beta_j^2)|\Psi_{j1}\rangle \langle \Psi_{j1}| - 2\alpha_j\beta_j(|\Psi_{j1}\rangle \langle \Psi_{j0}| - |\Psi_{j0}\rangle \langle \Psi_{j1}|).
\end{aligned} \tag{86}$$

We can see that this $G_j|_{\mathcal{H}}$ has the same structure as the analogous operator in [18]. By using the same proof of correctness of abs-QADC in [18] we can see that each $|\Psi_j\rangle_{\text{tr, tst, B}}$ can be decomposed into two eigenstates of G_j . Let $\alpha_j = \sin(\pi\theta_j)$ for $\theta_j \in [\frac{1}{4}, \frac{1}{2})$. Substituting this in Eq. (86), we get

$$G_j|_{\mathcal{H}} = \cos(2\pi\theta_j)(|\Psi_{j0}\rangle \langle \Psi_{j0}| + |\Psi_{j1}\rangle \langle \Psi_{j1}|) - \sin(2\pi\theta_j)(|\Psi_{j1}\rangle \langle \Psi_{j0}| - |\Psi_{j0}\rangle \langle \Psi_{j1}|). \tag{87}$$

We may write $G_j|_{\mathcal{H}}$ as

$$G_j|_{\mathcal{H}} = \begin{bmatrix} \cos(2\pi\theta_j) & \sin(2\pi\theta_j) \\ -\sin(2\pi\theta_j) & \cos(2\pi\theta_j) \end{bmatrix}. \quad (88)$$

This implies that G_j has eigenvectors

$$|\Psi_{j\pm}\rangle = \frac{1}{\sqrt{2}}(|\Psi_{j0}\rangle \pm i|\Psi_{j1}\rangle), \quad (89)$$

with eigenvalues $\Psi_{\pm} = e^{\pm i2\pi\theta_j}$ respectively. Now, we can decompose $|\Psi_j\rangle$ as

$$|\Psi_j\rangle = \frac{-i}{\sqrt{2}}(e^{i\pi\theta_j}|\Psi_{j+}\rangle - e^{-i\pi\theta_j}|\Psi_{j-}\rangle), \quad (90)$$

as required.

H Action of H as controlled H_j for the dot product based QkNN

Recall that

$$\begin{aligned} H &= V_{\text{in, data, B}} \mathcal{S}_{0\text{data, B}} \mathcal{V}_{\text{in, data, B}}^\dagger Z_B, \\ H_j &= (\mathbb{1} - 2|\Psi_j\rangle\langle\Psi_j|_{\text{data, B}})Z_B \end{aligned} \quad (91)$$

Then,

$$\begin{aligned} H &= V_{\text{in, data, B}} \left(\mathbb{1}_{\text{in, data, B}} - \mathbb{1}_{\text{in}} \otimes 2|0\rangle\langle 0|_{\text{data, B}} \right) V_{\text{in, data, B}}^\dagger \left(\mathbb{1}_{\text{in, data}} \otimes Z_B \right) \\ &= \left(\mathbb{1}_{\text{in, data, B}} - \sum_{k=0}^{M-1} 2|k\rangle\langle k|_{\text{in}} \otimes |\Psi\rangle\langle\Psi|_{\text{data, B}} \right) \left(\mathbb{1}_{\text{in, data}} \otimes Z_B \right) \\ &= \sum_{k=0}^{M-1} |k\rangle\langle k|_{\text{in}} \otimes \left(\mathbb{1}_{\text{data, B}} - 2|\Psi_j\rangle\langle\Psi_j|_{\text{data, B}} \right) \left(\mathbb{1}_{\text{in, data}} \otimes Z_B \right) \\ &= \sum_{k=0}^{M-1} |k\rangle\langle k|_{\text{in}} \otimes H_{k\text{data, B}}. \end{aligned} \quad (92)$$

I Eigenvectors of H_j for the dot product based QkNN

Recall that $H_j = (\mathbb{1} - 2|\Psi_j\rangle\langle\Psi_j|)Z_B$. Let

$$|\Psi_{j0}\rangle = \frac{1}{2\alpha_j}(|v\rangle_{\text{data}} + |u_j\rangle_{\text{data}})|0\rangle_B \quad \text{and} \quad |\Psi_{j1}\rangle = \frac{1}{2\beta_j}(|v\rangle_{\text{data}} - |u_j\rangle_{\text{data}})|1\rangle_B, \quad (93)$$

where,

$$\alpha_j = \sqrt{\frac{1}{2}(1 + X_j)} \quad \text{and} \quad \beta_j = \sqrt{\frac{1}{2}(1 - X_j)}. \quad (94)$$

So we may write,

$$|\Psi_j\rangle = \alpha_j|\Psi_{j0}\rangle + \beta_j|\Psi_{j1}\rangle. \quad (95)$$

Consider the subspace $\mathcal{H} = \text{span}(|\Psi_{j0}\rangle, |\Psi_{j1}\rangle)$. Then

$$Z_B|_{\mathcal{H}} = |\Psi_{j0}\rangle\langle\Psi_{j0}| - |\Psi_{j1}\rangle\langle\Psi_{j1}|. \quad (96)$$

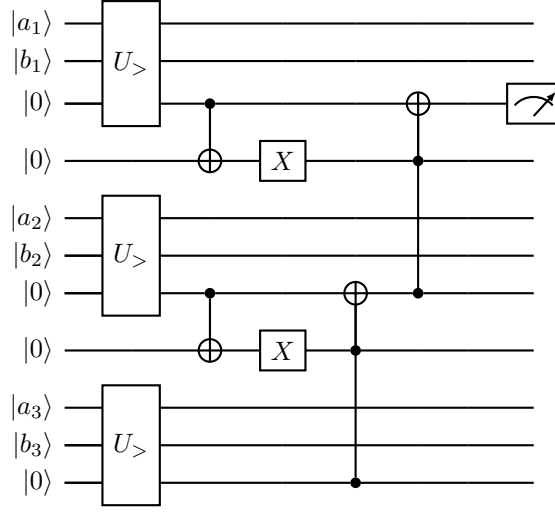


Figure 10: Circuit for implementing J gate for 3 qubit bit strings. Note that by replacing the $U_>$ with $U_{\neq}$ gates, we can implement $D^{(i)}$ gates.



Figure 11: Circuits used in bitstring comparison.

We have

$$\left(\mathbb{1} - 2|\Psi_j\rangle\langle\Psi_j| \right) \Big|_{\mathcal{H}} = (1 - 2\alpha_j^2)|\Psi_{j0}\rangle\langle\Psi_{j0}| + (1 - 2\beta_j^2)|\Psi_{j1}\rangle\langle\Psi_{j1}| - 2\alpha_k\beta_k(|\Psi_{j1}\rangle\langle\Psi_{j0}| + |\Psi_{j0}\rangle\langle\Psi_{j1}|). \quad (97)$$

This implies

$$H_j \Big|_{\mathcal{H}} = (1 - 2\alpha_j^2)|\Psi_{j0}\rangle\langle\Psi_{j0}| - (1 - 2\beta_j^2)|\Psi_{j1}\rangle\langle\Psi_{j1}| - 2\alpha_k\beta_k(|\Psi_{j1}\rangle\langle\Psi_{j0}| - |\Psi_{j0}\rangle\langle\Psi_{j1}|). \quad (98)$$

The result immediately follows by comparing Eq. (98) with Eq. (86).

J Bit-string comparison

In this section, we give a basic example of a circuit capable of comparing two-qubit bit string states. The J gate, mentioned in Eq. (11), for 1 qubit bit strings, can be implemented by the $U_>$ gate given in Fig. 11b.

Using $U_>$ as a subroutine, in Fig. 10 we give an example of a circuit capable of comparing 3-qubit bit strings $|a\rangle = |a_1\rangle|a_2\rangle|a_3\rangle$ and $|b\rangle = |b_1\rangle|b_2\rangle|b_3\rangle$.

This circuit compares the bits from left to right using the $U_>$ gate, that is, first a_3 and b_3 are compared, then a_2 and b_2 and so on. At any point, if $U_>$ returns 1, then the final output will be 1.

For executing the $D^{(i)}$ used in Eq. (13), we replace the $U_>$ gate with the $U_{\neq}$ gate given in Fig. 11a. This gate returns 1 if the input bits are unequal. By using $U_{\neq}$ in the circuit U , we can check, from left to right, if each pair of qubits are unequal or not. At any point, if we see an unequal pair of qubits, the circuit outputs 1, else 0. Different versions of these circuits and extension to n qubits are explained in [40].